Computation for Analytics

DIANE WOODBRIDGE, PH.D

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



Stands for Numerical Python

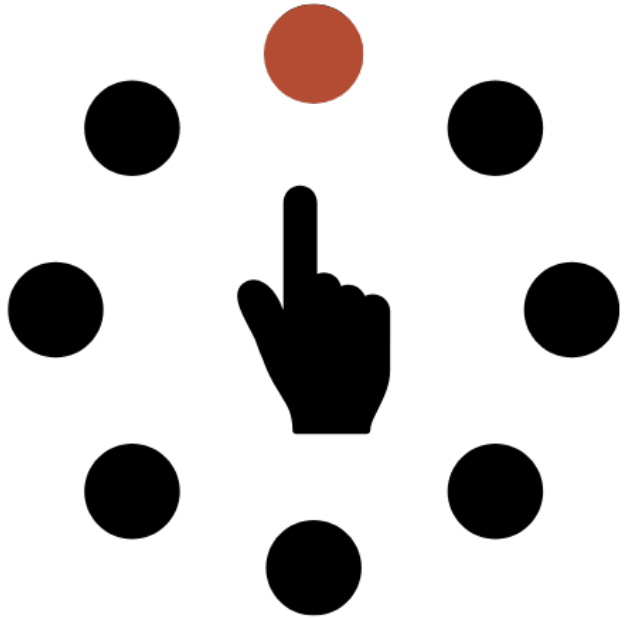
- Provide high-level functions for efficient array processing, mathematical functions, random number generator, linear algebra, Fourier transformation, etc.
- The NumPy API is used extensively in Pandas, SciPy, Matplotlib, scikit-learn, scikit-image and most other data science and scientific Python packages.

https://numpy.org/learn/

NumPy

It provides very extensive functions - I will cover ones that are frequently used.

- NumPy also has great documentation with many examples. (You can also use help() to see those).



https://numpy.org/learn/

Contents

Numpy

- **Installation**
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



NumPy Installation

```
$conda install –c anaconda numpy
```

https://anaconda.org/anaconda/numpy

NumPy



NumPy Array Creation

Indexing

Updating Data Types

Array Manipulation - Transpose, Reshape

Mathematical Methods

Create Custom Functions

Searching/Sorting/Unique

Linear Algebra

Random Numbers…

https://numpy.org/

Contents

Numpy

- Installation
- **NumPy Array Creation**
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



NumPy Array Creation

1. Converting Python sequences to NumPy arrays with optional dtype (indicating data type)

```
np.array(python_array[, dtype=float])
```

2. Reading from a file

```
np.loadtxt(fname=file_path,
           delimiter=‘,',
           ndim=n1,
           dtype=[(‘field_name', data_types),
                  ('field_name', data_types),...],
           skiprows=n2)
```

ex01-np_array.ipynb

https://numpy.org/doc/stable/user/basics.creation.html
https://numpy.org/doc/stable/reference/arrays.dtypes.html
https://numpy.org/doc/stable/reference/generated/numpy.loadtxt.html#numpy.loadtxt

Contents

Numpy

- Installation
- NumPy Array Creation
- **Indexing**
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



Indexing

Function	Description	Example
`array[row][col] or array[row, col]`	**Single Element Indexing** - returns the data located at the given row and col	`np_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])` `np_array[:, 2] # returns array([3, 6, 9])`
`array[start:stop:step] [start:stop:step]`	**Slicing and striding** - returns the data located from start upto stop using step as a stride.	`np_array[0:2:1, 1:4:2] # returns array([[2],[5]])`
`array[array_condition]`	**Boolean Indexing** - returns the data that meets the array_condition.	`np_array[np_array[:, 0] < 5]] # returns array([[1, 2, 3],[4, 5, 6]])`
`array[[row_indices], [col_indices]]`	**Fancy Indexing** - selects a subset of rows in a particular order, you can simply pass a list or ndarray of integers indicating the desired order.	`np_array[[2, 0]] # returns array([3, 7])`

ex02-indexing.ipynb

https://numpy.org/doc/stable/user/basics.indexing.html

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- **Updating Data Types**
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



Updating Data Types

- You can check your NumPy data type by `array.dtype`.
- `array.astype(dtype)` : Create an array by converting array to dtype
  - `dtype`
    - By default, the data-type is inferred from the input data.
    - In addition to Python data types, you can use the following array-protocol type strings.
      - Ex. ‘U20’ means 20-character string
        bootcamp.astype(‘U20') returns
        array(['Dine', 'Shan'], dtype=‘<U20')

Array-protocol type strings	Details
'?'	boolean
'b'/'B'	Signed/unsigned byte
'i'/'u'	Signed/unsigned integer
'f'	floating-point
'U'	Unicode string
'O'	(Python) objects
'M'	datetime

ex03-data_types.ipynb
https://numpy.org/doc/stable/reference/arrays.dtypes.html

Contents

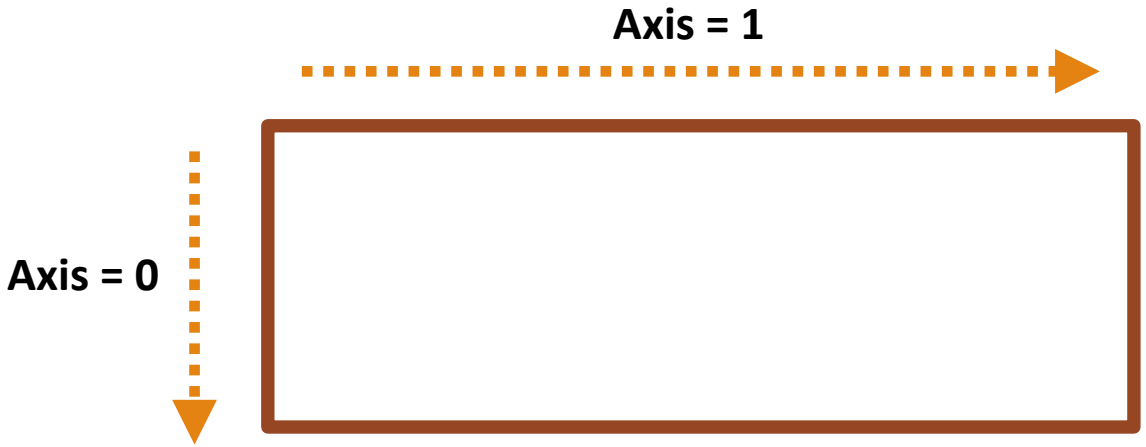
Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- **Array Manipulation - Transpose, Reshape**
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy

Array Manipulation

- Axis in Numpy



NumPy



Array Manipulation

Function	Description	Example
`np.transpose(array, axes=None)`	Returns the transposed array	`np_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])` `np.transpose(np_array) # returns array([[1, 4, 7], [2, 5, 8], [3, 6, 9]])`
`np.reshape(array, int/ tuple of int)`	Returns a new shape to an array without changing its data	`np.reshape(np_array, [9, 1]) # returns array([[1], [2], [3], [4], [5], [6], [7], [8], [9]])`
`np.split(array, indices_or_sections, axis)`	Split an array into multiple sub-arrays	`np.split(np_array, [1, 2], 0) # returns [array([[1, 2, 3]]), array([[4, 5, 6]]), array([[7, 8, 9]])]`
`np.concatenate((array_1, array_2), axis)`	Join a sequence of arrays along a new axis.	`col = np.array([[10], [11], [12]])` `np.concatenate((np_array, col), 1) #returns array([[ 1, 2, 3, 10],[4, 5, 6, 11], [7, 8, 9, 12]])`

ex04-array_manipulation.ipynb

https://numpy.org/doc/stable/reference/routines.array-manipulation.html

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- **Mathematical Methods**
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

NumPy



Mathematical Methods

Function	Description	Example

`np.exp/log(array)`	Return the exponent/logarithm value of each element.	`arr_1 = np.array([1, 2, 3])`
		`arr_2 = np.array([-1, 5, 6])`
		`np.exp(arr_1). # returns [ 2.718 7.389  20.086]`

`np.ceil(array)/floor(array)/around(array, decimal)`	Return the ceiling/floor/round of each element.	`np.ceil(arr_1)  # returns [1. 2. 3.]`
`np.min/max/sum/mean/median/std(array)`	Return minimum, maximum, sum, mean, median, standard deviatio value in array	`np.min(arr_1)  # returns 1`
`np.argmin/argmax(array)`	Return indices of minimum/maximum value.	`np.argmin(arr_1)  # returns 0`
`np.cov(array_1, array_2)`	Estimate a covariance matrix. The covariance matrix element Cij is the covariance of array_i and array_j.	`np.cov(arr_1, arr_2)  # returns array([[ 1., 3.5], [ 3.5, 14.333]])`
`np.add/subtract/multiply/divide/maximum/minimum (array_1, array_2)`	Return an array by adding/subtracting/multiplying/dividing/comparing elements in array_1 and array_2.	`np.add(arr_1, arr_2))  # returns [0 7 9]` `np.minimum(arr_1, arr_2)  # returns [-1  2  3]`
`np.histogram(array, bins)`	Compute the histogram of array.	`arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])` `np.histogram(arr, 3)  # returns (array([3, 3, 4]), array([ 1., 4., 7., 10.]))`
`np.percentile(array, q)`	Compute the q-th percentile of array.	`np.percentile(arr, [10, 90])  # returns array([1.9, 9.1])`

https://numpy.org/doc/stable/reference/routines.math.html

ex05-math.ipynb

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- **Custom Functions**
- Searching/Sorting/Unique
- Linear Algebra

NumPy



Creating Custom Functions

- **`numpy.vectorize(function)`** can create a function that can be apply to an array(s).
  - Ex.

```
def myfunc(a, b):
      if a > b:
          return a
       else:
          return b

vfunc = np.vectorize(myfunc)
vfunc([1,2,3],[0,3,4]) # returns [1,3,4]
```

ex06-custom_functions.ipynb

https://numpy.org/doc/stable/reference/generated/numpy.vectorize.html

Example 06

Create a function, called string_len() which returns the length of string in a numpy array.

Create a function, called string_len_compare() which returns a longer string of the same location in two element in two input numpy arrays.

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- **Searching/Sorting/Unique**
- Linear Algebra

NumPy



Searching/Sorting/Unique

Function	Description	Example
`np.where(condition, x, y)`	Searching : You can use boolean indexing covered in Indexing for only returning values that satisfy the given condition. x will be returned when condition is True, otherwise y.	`nums = np.array(range(10))` `np.where(nums < 5, nums*10, nums) # returns array([0, 10, 20, 30, 40, 5, 6, 7, 8, 9])`

`np.sort(array, axis)`	Return a sorted array by axis.	`array = np.array([[1, 1, 2, 2],`
		`[2, 2, 3, 3],`
		`[2, 2, 1, 1]])`
		`np.sort(array, axis=1) # returns array([[1, 1, 2, 2],`
		`[2, 2, 3, 3],`
		`[1, 1, 2, 2]])`

`np.unique(array)`	Set routine, Returns the sorted unique elements of an array	`np.unique(array) # returns array([1, 2, 3])`

ex07-search_sort_unique.ipynb

https://numpy.org/doc/stable/reference/generated/numpy.where.html

https://numpy.org/doc/stable/reference/generated/numpy.unique.html

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- **Linear Algebra**

NumPy



Linear Algebra

Function	Description	Example

`np.dot(array_1, array_2)`	Matrix multiplication of two arrays.	`array_1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])`
		`array_2 = np.array([[1, 2], [3, 4], [5, 6]])`
		`np.dot(array_1, array_2)  # returns array([[ 22, 28], [ 49,  64], [ 76, 100]])`

`np.linalg.det(array)`	Compute the determinant of array.	`np.linalg.det(array_1)  # returns -9.5162e-16`
`np.linalg.eig(array)`	Compute the eigenvalues and eigenvectors of a square matrix	`np.linalg.eig(array_1)  # returns (array([ 1.6117e+01, -1.1168e+00, -3.3843e-16]), array([[-0.23197, -0.7858,  0.4082],[-0.5253, -0.08675, -0.81658],[-0.8187,  0.6123,  0.4082]]))`
`np.linalg.inv(array)`	Compute the inverse of a matrix.	`np.linalg.inv(array_1)  # returns array([[ 3.1525e+15, -6.3050+15,  3.1525e+15] [-6.3050e+15,  1.2610e+16, -6.30504e+15], [ 3.1525e+15, -6.3050e+15,  3.1525e+15]])`
`np.linalg.svd(array)`	Compute the Singular Value Decomposition (SVD).	`np.linalg.svd(array_1)  # returns (array([[-0.2148, 0.8872,  0.4082], [-0.5206,  0.2496, -0.8165, [-0.8263, -0.3879,  0.4082]]), array([1.6848e+01, 1.0684e+00, 3.3348e-16]), array([[-0.4797, -0.5724, -0.6651], [-0.7767, -0.0757,  0.6253], [-0.4082, 0.8165, -0.4082]]))`

ex08-linear_algebra.ipynb

https://numpy.org/doc/stable/reference/routines.linalg.html

Example 9

Create a function called sorted_filtered_rows() which takes a numpy array created from countries.tsv, starts_with and lon_limit, and returns names in lower cases where its name starts with start_with (case insenstive) and its longitude is smaller than lon_limit.

- Data/countries.tsv contains a tab separated values of initial, name, lon, and lat.
- Example.

```
sorted_filtered_rows(countries, "a", 0)  # returns array(['american samoa', 'angola', ‘antarctica', 'argentina', 'australia'])
```

Contents

Numpy

- Installation
- NumPy Array Creation
- Indexing
- Updating Data Types
- Array Manipulation - Transpose, Reshape
- Mathematical Methods
- Custom Functions
- Searching/Sorting/Unique
- Linear Algebra

References

Python : https://docs.python.org/3/

NumPy : https://numpy.org/